Partie 1

Imports

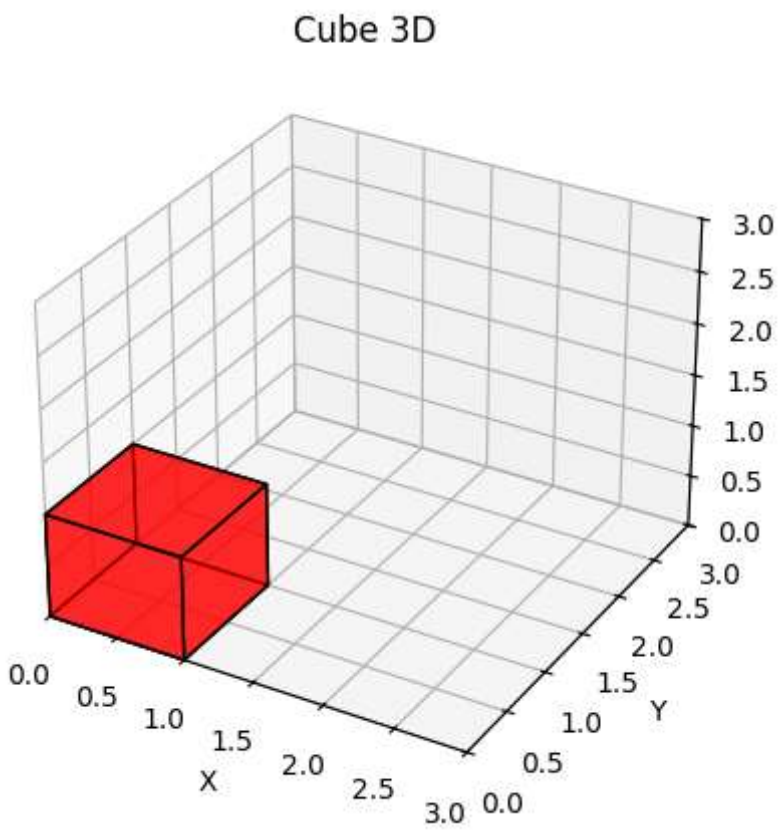
In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

Définition des points du cube et création de la fonction get_faces

In [2]:
```
points = np.array([
    [0, 0, 0],
    [1, 0, 0],
    [1, 1, 0],
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 1],
    [1, 1, 1],
    [0, 1, 1]
])

def get_faces(points):
    return [
        [points[0], points[1], points[2], points[3]],  # ABCD
        [points[4], points[5], points[6], points[7]],  # EFGH
        [points[0], points[1], points[5], points[4]],  # ABFE
        [points[2], points[3], points[7], points[6]],  # CDHG
        [points[1], points[2], points[6], points[5]],  # BCGF
        [points[4], points[7], points[3], points[0]]   # EHDA
    ]
```

Affichage du cube 3d

In [3]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

faces = get_faces(points)
ax.add_collection3d(Poly3DCollection(
    faces, facecolors='red', linewidths=1, edgecolors='black', alpha=.6
))

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("Cube 3D")

ax.set_xlim([0, 3])
ax.set_ylim([0, 3])
ax.set_zlim([0, 3])

plt.show()
```

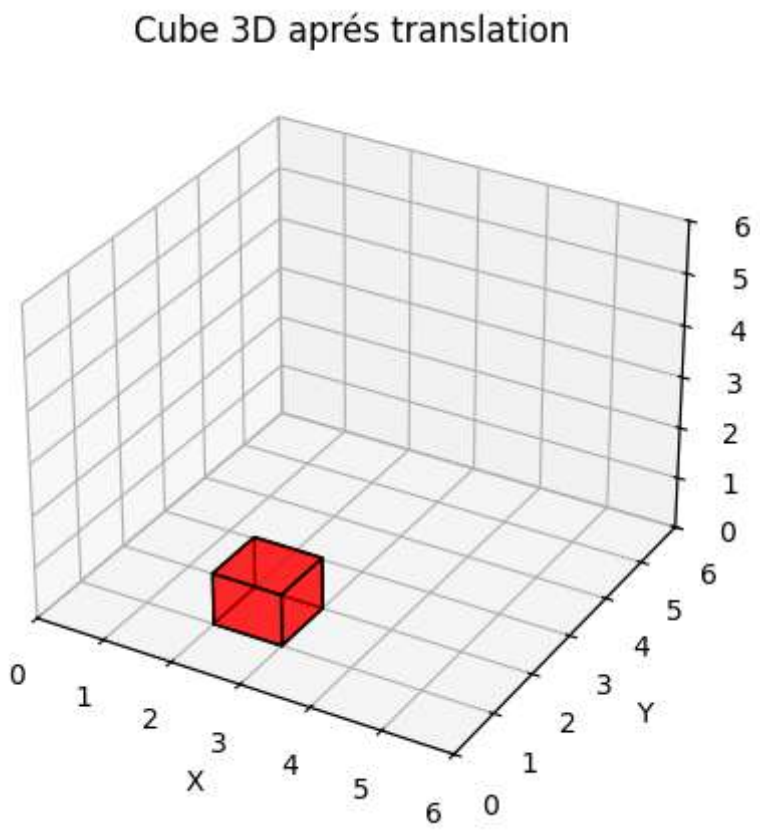


Translation du Cube

La translation consiste à déplacer le cube dans l'espace sans modifier sa forme ni son orientation.

La translation est réalisée en ajoutant un vecteur de translation **T** à chaque point du cube :

In [4]:
```
T = np.array([2, 1, 0])
translatedPoints = points + T

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

translatedFaces = get_faces(translatedPoints)

ax.add_collection3d(Poly3DCollection(
    translatedFaces, facecolors='red', linewidths=1, edgecolors='black', alpha=.
))

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("Cube 3D aprés translation")

ax.set_xlim([0, 6])
ax.set_ylim([0, 6])
ax.set_zlim([0, 6])

plt.show()
```



Mise à l'échelle

La mise à l'échelle permet de modifier la taille du cube.

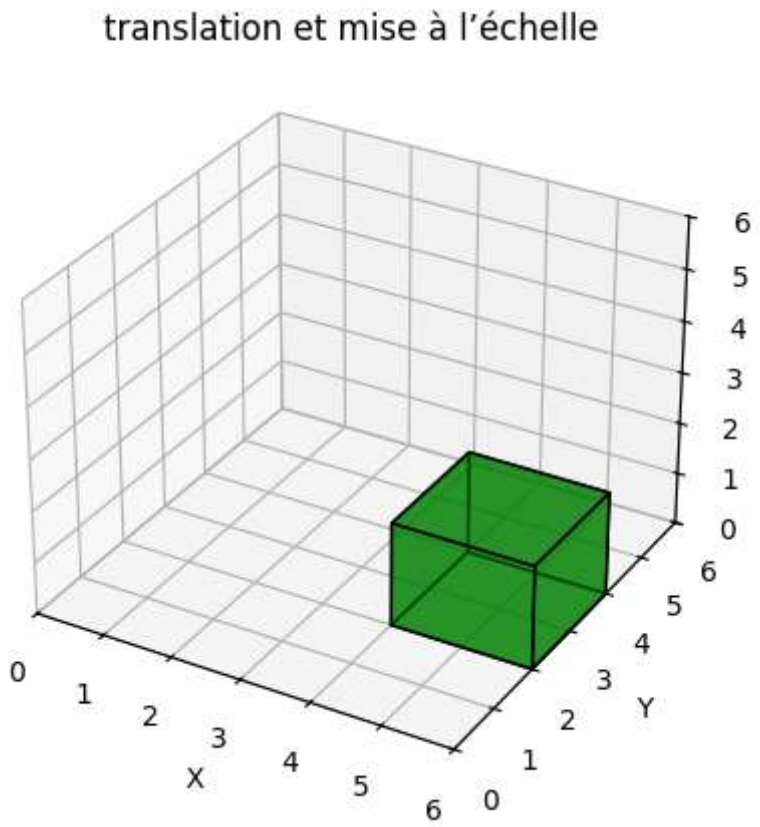
Cette transformation est effectuée à l'aide d'une matrice de mise à l'échelle (S) :

P' = P x S

Selon les valeurs de la matrice, le cube peut être agrandi ou rétréci uniformément ou selon un axe spécifique.

In [5]:
```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

S = np.array([
    [2, 0, 0],
    [0, 2, 0],
    [0, 0, 2]
])
ScaledPoints = translatedPoints @ S.T
faces_scaled = get_faces(ScaledPoints)

ax.add_collection3d(Poly3DCollection(
    faces_scaled, facecolors='green', linewidths=1, edgecolors='black', alpha=0.
))

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("translation et mise à l'échelle")

ax.set_xlim([0, 6])
```

```
ax.set_ylim([0, 6])
ax.set_zlim([0, 6])

plt.show()
```



Rotation

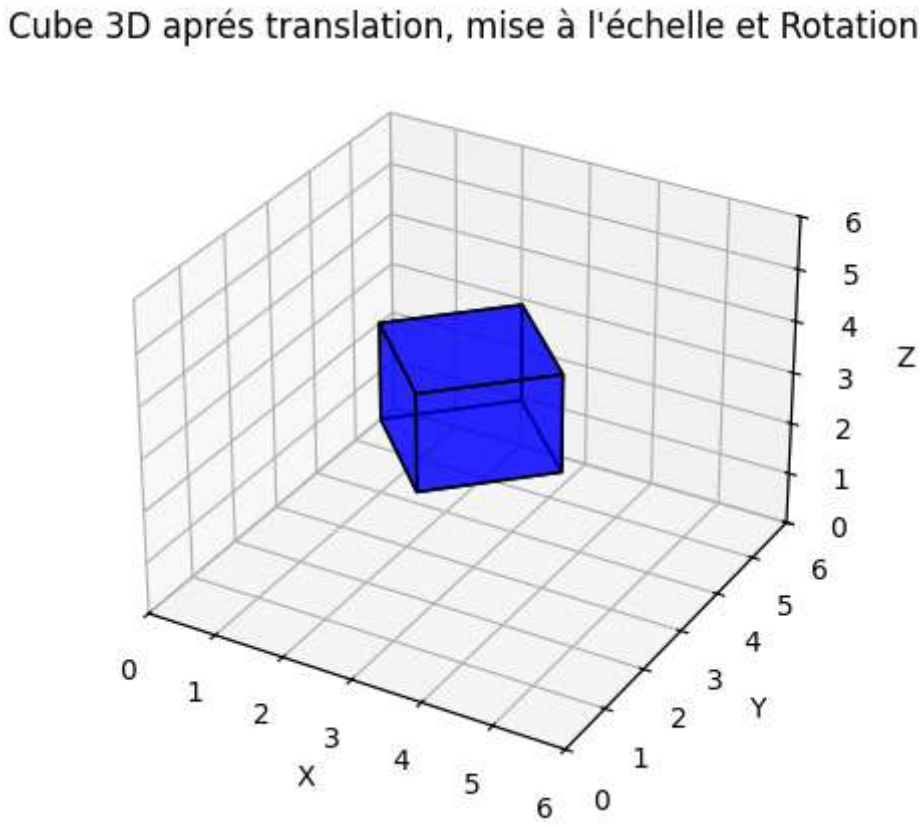
La rotation permet de faire tourner le cube autour d'un axe donné.

Lorsqu'une rotation est appliquée :

- les distances entre les sommets du cube restent inchangées,
- la forme du cube est conservée,
- seule l'orientation du cube dans l'espace est modifiée.

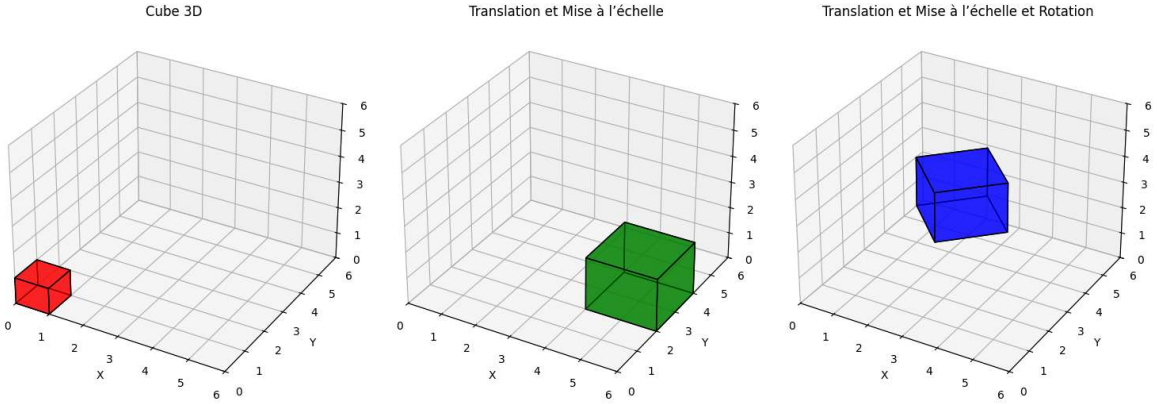
In [6]:
```
theta = np.pi / 4

Rz = np.array([
    [np.cos(theta), -np.sin(theta), 0],
    [np.sin(theta),  np.cos(theta), 0],
    [0,              0,             1]
])

RotatedPoints = ScaledPoints @ Rz.T


fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

RotatedFaces = get_faces(RotatedPoints)

ax.add_collection3d(Poly3DCollection(
```

```
    RotatedFaces, facecolors='blue', linewidths=1, edgecolors='black', alpha=0.6
))

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title("Cube 3D aprés translation, mise à l'échelle et Rotation")

ax.set_xlim([0, 6])
ax.set_ylim([0, 6])
ax.set_zlim([0, 6])

plt.show()
```



Partie 2

Les transformations géométriques **ne sont pas commutatives** effectuer une rotation puis une translation ne donne pas le même résultat que effectuer une translation puis une rotation.

In [7]:
```
def DessinerCube(ax, points, title, color):
    faces = get_faces(points)
    ax.add_collection3d(Poly3DCollection(
        faces, facecolors=color, linewidths=1, edgecolors='black', alpha=0.6
    ))
    ax.set_title(title)
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_xlim([0, 6])
    ax.set_ylim([0, 6])
```

```
    ax.set_zlim([0, 6])

fig = plt.figure(figsize=(15, 5))

ax1 = fig.add_subplot(131, projection='3d')
DessinerCube(ax1, points, "Cube 3D", "red")

ax2 = fig.add_subplot(132, projection='3d')
DessinerCube(ax2, ScaledPoints, "Translation et Mise à l’échelle", "green")

ax3 = fig.add_subplot(133, projection='3d')
DessinerCube(ax3, RotatedPoints, "Translation et Mise à l’échelle et Rotation",

plt.tight_layout()
plt.show()
```